

STACK TECNOLÓGICO Y METODOLOGÍA DE DESARROLLO

Propuesta técnica para el desarrollo de una plataforma integral de venta de entradas, control de acceso y gestión de eventos.

Proyecto	Plataforma de Ticketing y Control de Acceso para Eventos
Documento	Stack tecnológico, arquitectura y flujo de trabajo de desarrollo
Fecha	25 de septiembre de 2026

1. Introducción

Este documento describe la arquitectura técnica, el stack tecnológico y la metodología de trabajo que se aplicarán para el desarrollo de la plataforma. El objetivo es que cualquier persona involucrada en el proyecto — técnica o no técnica — entienda con qué herramientas se construye el sistema, por qué se eligió cada una, y cómo se organiza el trabajo de desarrollo de principio a fin.

La plataforma se plantea como un ecosistema único: un sitio público de venta de entradas y un panel de administración para organizadores, ambos conectados a la misma base de datos central, evitando duplicar información entre sistemas independientes.

2. Arquitectura de la solución

La solución se plantea como **dos aplicaciones web independientes** que comparten la misma base de datos, en lugar de un único sistema monolítico:

- **Sitio de compra (marketplace):** la cara pública de la plataforma — exploración de eventos, selección de entradas o asientos, checkout de pago.
- **Panel de organizador:** gestión de eventos, entradas, precios, mapas de asientos, control de la cola virtual y reportes de venta.

Separar ambas aplicaciones — con inicios de sesión y sesiones completamente independientes entre sí — reduce la superficie de ataque de cada una, evita que un problema en el sitio público comprometa el panel administrativo, y permite escalar o desplegar cada parte de forma independiente.

Organización del código: monorepo

Ambas aplicaciones vivirán dentro de un mismo repositorio de código (**monorepo**), organizadas en workspaces independientes, con un paquete compartido para el acceso a la base de datos. Esto permite versionar todo el sistema en conjunto, reutilizar código común (modelos de datos, validaciones) y mantener consistencia entre ambas aplicaciones sin duplicar lógica.

3. Stack tecnológico

Resumen de las tecnologías que se utilizarán en cada capa del sistema:

Capa	Tecnología	Rol en el proyecto
Lenguaje	TypeScript	Lenguaje principal de todo el proyecto (frontend, backend y scripts). Tipado estático que reduce errores en tiempo de desarrollo, especialmente relevante en un sistema que maneja pagos e inventario de entradas.
Framework web	Next.js (React)	Framework full-stack: renderiza las páginas en el servidor para mejor rendimiento y posicionamiento (SEO), y maneja también la lógica de backend (API y acciones de servidor) dentro del mismo proyecto.
Interfaz / estilos	Tailwind CSS	Sistema de diseño basado en utilidades, permite construir interfaces consistentes y responsive (celular, tablet, escritorio) de forma ágil.
Base de datos	PostgreSQL	Motor de base de datos relacional, robusto y probado en producción, adecuado para datos transaccionales como ventas, inventario de entradas y usuarios.
Acceso a datos	Prisma ORM	Capa de acceso a la base de datos con tipado seguro extremo a extremo: los modelos de datos definidos en un solo lugar se reflejan automáticamente en el código de la aplicación, reduciendo errores de consistencia.
Autenticación	NextAuth (sesiones JWT)	Manejo de inicio de sesión con credenciales propias, con sesiones independientes por aplicación (comprador vs. organizador).
Pagos	Pasarela de pago local (a definir con el cliente)	Integración vía preferencias de pago y webhooks: el estado del pago se confirma siempre del lado del servidor, nunca solo del navegador, para evitar fraude.
Cola virtual / alta demanda	Redis	Estructura de datos en memoria de alta velocidad, usada para manejar salas de espera y turnos cuando la demanda de un evento supera la capacidad de venta simultánea.
Control de versiones	Git	Historial completo y trazable de cada cambio realizado sobre el código.
Infraestructura	Hosting en la nube (Vercel / proveedor equivalente) + base de datos administrada	Despliegue con entornos separados de desarrollo, pruebas y producción.

4. Por qué este stack

- **Un solo lenguaje de punta a punta (TypeScript):** el mismo lenguaje se usa en la interfaz, el backend y los scripts de base de datos. Esto reduce el tiempo de incorporación de nuevos desarrolladores al proyecto y evita errores de comunicación entre capas.
- **Tipado estático:** los errores de datos (por ejemplo, un precio mal tipado o un campo faltante) se detectan al escribir el código, no cuando el sistema ya está en producción.
- **Tecnologías maduras y ampliamente adoptadas:** Next.js, PostgreSQL, Prisma y Redis son herramientas con comunidades grandes, documentación extensa y soporte a largo plazo — no dependemos de tecnologías experimentales ni de un solo proveedor.
- **Preparado para alta demanda:** el diseño contempla desde el inicio los picos de tráfico propios de la venta de entradas (una sala de espera virtual y bloqueo de inventario a nivel de base de datos previenen la sobreventa de entradas o asientos).

5. Flujo de trabajo de desarrollo

5.1 Control de versiones

Todo el código fuente se gestiona con Git. Cada funcionalidad nueva o corrección se desarrolla en una rama independiente y se integra a la rama principal mediante revisión de código, manteniendo siempre un historial claro y reversible de cada cambio.

5.2 Entornos

- **Desarrollo (local):** entorno de trabajo diario, con datos de prueba, aislado de cualquier dato real.
- **Staging / pruebas:** entorno idéntico a producción donde se valida cada entrega antes de que el cliente la revise, usando credenciales de prueba (pasarela de pago en modo sandbox, por ejemplo).
- **Producción:** entorno final con datos y transacciones reales.

5.3 Ciclo de desarrollo iterativo

El desarrollo se organiza por módulos o entregas incrementales en lugar de un único bloque monolítico al final del proyecto. Cada entrega se puede revisar y probar de forma independiente, permitiendo ajustar el rumbo temprano si algo no cumple lo esperado, en vez de descubrirlo recién al final.

5.4 Calidad y pruebas

- Verificación de tipos y de estilo de código automatizada antes de integrar cualquier cambio.
- Pruebas manuales guiadas de cada flujo crítico (compra, pago, emisión de entrada, control de acceso) antes de cada entrega.
- Revisión de código: ningún cambio se integra sin una segunda revisión.

5.5 Documentación

Se mantiene documentación técnica viva del proyecto (arquitectura, modelo de datos, decisiones de diseño y guía de instalación), de forma que cualquier persona pueda incorporarse al desarrollo sin depender del conocimiento acumulado de una sola persona.

6. Seguridad

- Contraseñas nunca almacenadas en texto plano (hash con algoritmo estándar de la industria).
- Sesiones de comprador y de organizador completamente separadas — un acceso comprometido en un lado no expone al otro.
- Todo pago se confirma del lado del servidor mediante webhook firmado por la pasarela, nunca se confía únicamente en la respuesta del navegador del comprador.
- Bloqueo de inventario (entradas o asientos) a nivel de base de datos para evitar que dos compradores adquieran el mismo cupo simultáneamente.
- Validación de datos de entrada en cada operación sensible del servidor.
- Control de acceso por rol: cada perfil (comprador, organizador, operador) solo puede ver y hacer lo que le corresponde.

7. Escalabilidad y rendimiento

- Renderizado del lado del servidor para tiempos de carga rápidos incluso en conexiones móviles limitadas.
- Sala de espera virtual (cola) para eventos de alta demanda: en lugar de que el sitio colapse con miles de usuarios simultáneos, se los ordena en una fila con turno visible y se admite a la venta según la capacidad real del sistema.
- Base de datos e infraestructura pensadas para escalar horizontalmente a medida que crece el volumen de eventos y usuarios.
- Imágenes y contenido estático servidos de forma optimizada (compresión y tamaños adaptados a cada dispositivo).

8. Fases del proyecto

Plan de trabajo propuesto, organizado en fases incrementales:

Fase	Objetivo	Duración estimada
Fase 1 — Base del sistema	Arquitectura, modelo de datos, autenticación, gestión de eventos y entradas, landing pública.	—
Fase 2 — Compra y asientos	Carrito de compra, selección de asientos con mapa visual, checkout e integración de pago.	—
Fase 3 — Alta demanda	Sala de espera / cola virtual para eventos de alta demanda, endurecimiento del sistema para producción.	—
Fase 4 — Control de acceso	Emisión de entrada digital con QR, validación de acceso en puerta.	—
Fase 5 — Estabilización	Pruebas integrales, ajustes de rendimiento y seguridad, puesta en producción.	—

La duración de cada fase se definirá una vez acordado el alcance final con el cliente.

9. Consideraciones finales

Este stack y metodología priorizan tres cosas: un sistema que se pueda mantener y hacer crecer en el tiempo sin depender de una sola persona, un flujo de pagos e inventario confiable desde el primer día, y una experiencia de compra que se sostenga incluso bajo alta demanda. Cualquier ajuste de alcance o de tecnología se documentará y se reflejará en una actualización de este mismo documento.